CMR UNIVERSITY
Private University Established in Karnataka State by Act No. 45 of 2013

CAPSTONE PROJECT • AI & ML • 2026–27

CodeArchaeologist

An AST-Guided, LLM-Powered System for Legacy Code Comprehension & Automated Documentation

TEAM MEMBERS

Student 1 Name (USN: _____)
Student 2 Name (USN: _____)

GUIDE

Professor Name

Department of CSE, School of Engineering & Technology

Presentation Outline

1 Problem Area

2 Project Statement

3 Objectives

4 Literature Survey

5 Proposed Methodology

6 System Architecture

7 Data Flow Diagram

8 Technology Stack

9 System Modules

10 Expected Outcomes

11 Applications

12 Timeline & Conclusion

Problem Area

Comprehension is the bottleneck

Developers spend most of their time reading and understanding existing code rather than writing new code.

Undocumented legacy systems

Old codebases are poorly documented and the engineers who wrote them are often no longer available.

Slow, error-prone tracing

Manually following logic across large codebases delays onboarding and increases the risk of defects.

No plain-language insight

Existing tools describe code only structurally or numerically — they cannot explain what the code does or why.

Project Statement

Bridging structure and meaning

Static analysis for precision + LLMs for explanation = grounded, trustworthy documentation.

Modern software development rarely begins from scratch; developers spend most of their time reading, understanding, and modifying code that already exists. In large or ageing systems this code is frequently undocumented, and the engineers who originally wrote it may no longer be available. As a result, developers must manually trace logic across many files to answer even simple questions — such as what a module does or how components depend on one another. This is a well-known bottleneck that slows onboarding, delays development, and increases the likelihood of defects.

Traditional static-analysis tools extract accurate structure — abstract syntax trees, call graphs, and complexity metrics — but cannot explain code in natural language. Large Language Models explain code fluently, yet used alone they may hallucinate and struggle with large repositories. CodeArchaeologist combines both: it parses a codebase, builds a structured representation, and uses that grounded context to generate reliable explanations, documentation, architecture diagrams, and natural-language answers about the code.

Project Objectives

Code Comprehension

Generate plain-English explanations of files, classes, and functions.

Architecture Visualization

Build call graphs and dependency diagrams of the system.

Automated Documentation

Produce module- and function-level docs directly from source.

Code-Quality Analysis

Compute complexity metrics and detect dead code.

Natural-Language Search

Answer questions about the codebase using semantic search.

Grounded, Trustworthy AI

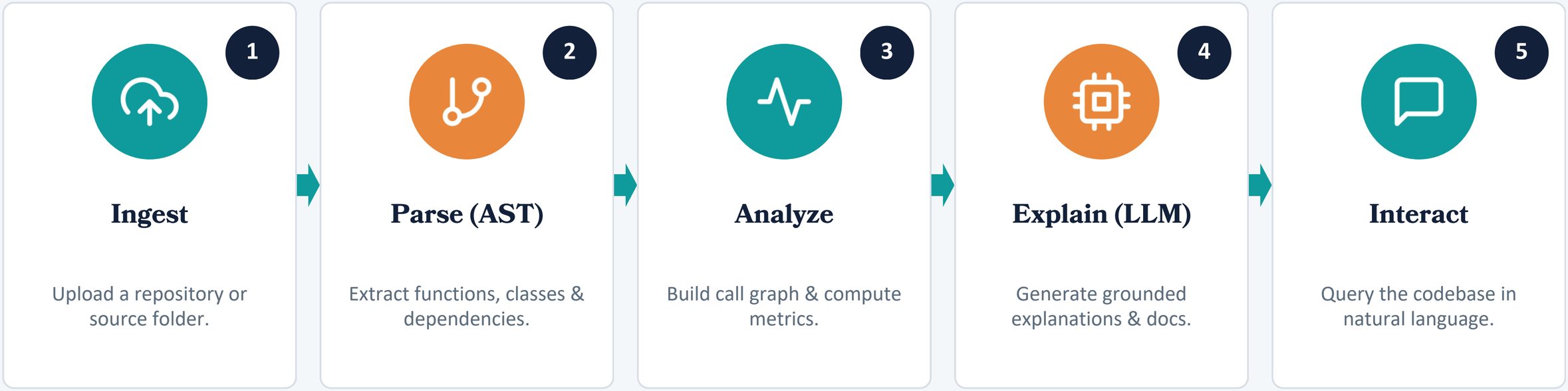
Feed the LLM structured analysis to reduce hallucination.

Literature Survey

Ref / Year	Work	Key Contribution
Feng et al., 2020	**CodeBERT**	Pre-trained model bridging code and natural language.
Chen et al., 2021	**Codex**	LLMs trained on code for generation and comprehension.
Guo et al., 2021	**GraphCodeBERT**	Adds data-flow structure to code representations.
Hu et al., 2018	**Deep Code Comment Gen.**	Automatic comment/documentation generation.
Ahmad et al., 2020	**Code Summarization**	Transformer-based source-code summarization.
Allamanis et al., 2018	**ML for Big Code (Survey)**	Foundations of learning over source code.

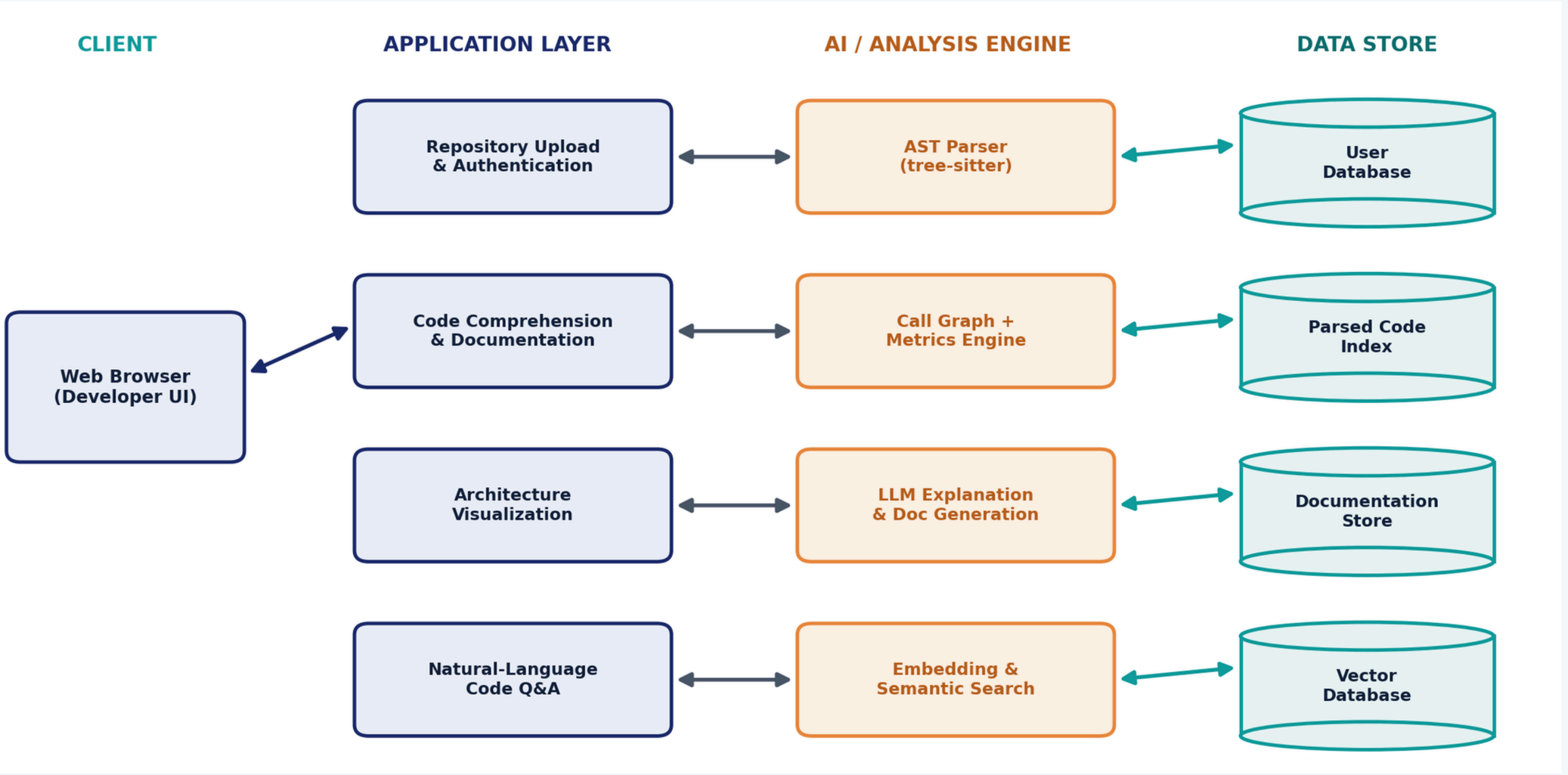
Proposed Methodology

CMR UNIVERSITY

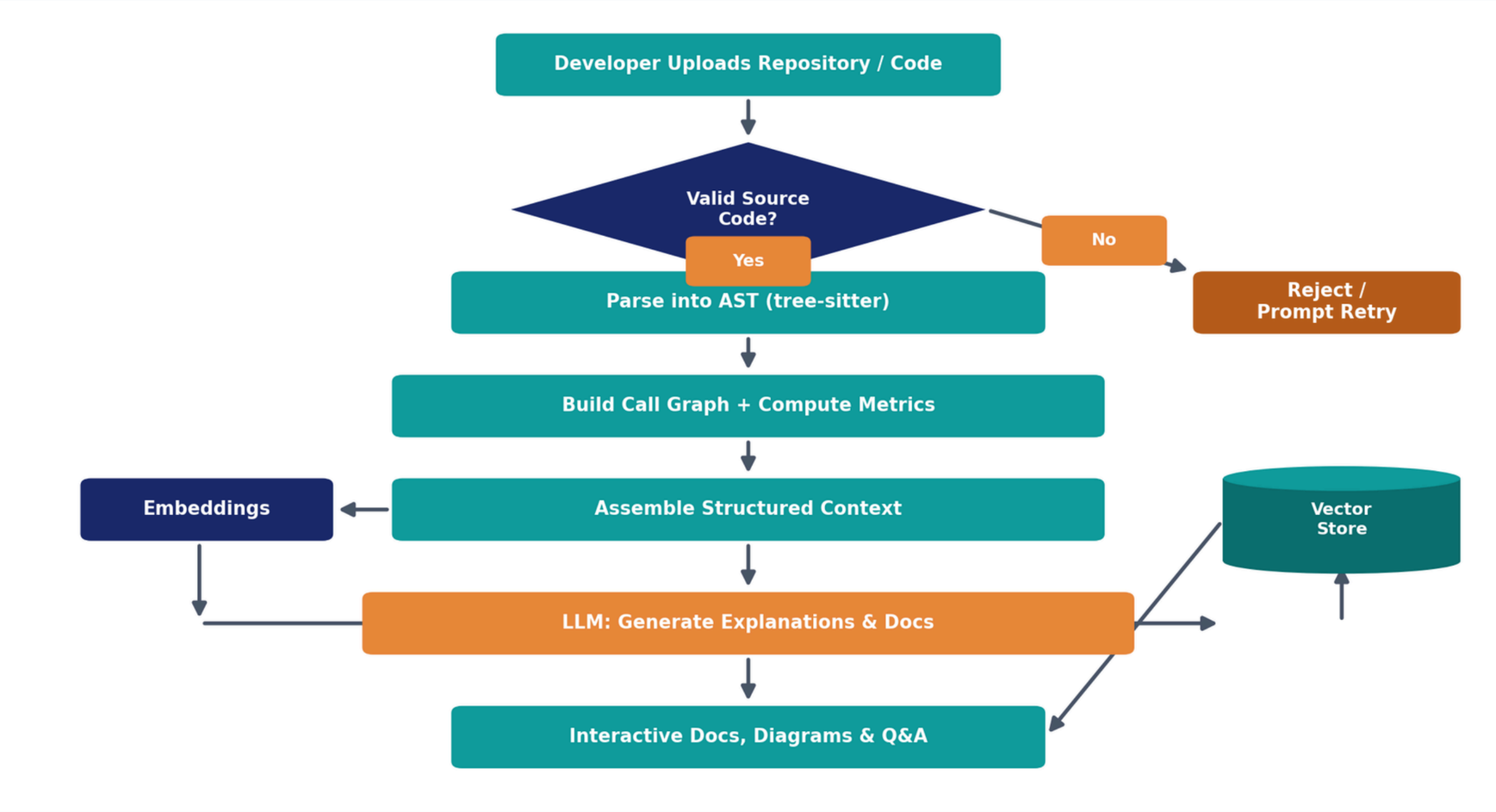


Static analysis supplies clean, structured context to the LLM — the key to grounded, low-hallucination output.

System Architecture



Data Flow Diagram



Technology Stack

CMR UNIVERSITY

Python

Core language

tree-sitter

AST parsing

NetworkX

Call graph & metrics

LLM (Claude/GPT)

Explanation & docs

Sentence-Transformers

Embeddings

FAISS / ChromaDB

Vector search

FastAPI

Backend API

React + Mermaid.js

Frontend & diagrams

System Modules

Module 1

Code Ingestion

Accepts a repo/folder and prepares files for analysis.

Module 2

Static Analysis

Parses code into ASTs and resolves dependencies.

Module 3

Metrics & Quality

Call graph, complexity, and dead-code detection.

Module 4

LLM Explanation

Generates grounded explanations and documentation.

Module 5

Visualization

Renders interactive architecture & dependency diagrams.

Module 6

Semantic Q&A

Embedding-based natural-language code search.

Expected Outcomes & Advantages

Expected Outcomes

- Plain-language documentation for any supported codebase.
- Interactive architecture and call-graph diagrams.
- Complexity and dead-code reports for maintainability.
- Natural-language Q&A over the codebase.
- Reduced onboarding time for new developers.

Grounded

AI output verified against real code structure, lowering hallucination.

Faster

Cuts the time spent understanding unfamiliar code.

Always current

Docs regenerate as the code changes — never stale.

Applications

Developer Onboarding

Helps new engineers understand a codebase in days, not weeks.

Enterprise Maintenance

Documents and de-risks large, undocumented legacy systems.

Education

Assists students in reading and learning from real-world code.

Code Audits & Review

Surfaces structure, complexity, and dead code for reviewers.

Project Timeline

Semester 1 — Analysis Engine

1. Requirement study & literature survey
2. AST parsing (tree-sitter) for Python
3. Call graph, complexity & dead-code metrics
4. Command-line prototype & testing

Semester 2 — AI & Interface

1. LLM explanation & documentation layer
2. Embeddings + semantic code Q&A
3. Web UI with architecture diagrams
4. Integration, evaluation & deployment

Conclusion & Future Scope

Conclusion

CodeArchaeologist unites classical program analysis with modern LLMs to turn opaque, undocumented codebases into clear explanations, living documentation, and searchable knowledge. By grounding AI in verifiable structure, it delivers reliable insight — reducing comprehension effort and improving maintainability across real-world software.

Future Scope

- Support for more languages (Java, C++, JS).
- IDE plugins for in-editor explanations.
- Automated hallucination & faithfulness evaluation.
- Team knowledge base with change-aware docs.

References

[1] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” Findings of EMNLP, pp. 1536–1547, 2020.

[2] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.

[3] D. Guo et al., “GraphCodeBERT: Pre-training Code Representations with Data Flow,” ICLR, 2021.

[4] X. Hu, G. Li, X. Xia, D. Lo, Z. Jin, “Deep Code Comment Generation,” IEEE/ACM ICPC, pp. 200–210, 2018.

[5] W. U. Ahmad, S. Chakraborty, B. Ray, K. Chang, “A Transformer-based Approach for Source Code Summarization,” ACL, pp. 4998–5007, 2020.

[6] M. Allamanis, E. T. Barr, P. Devanbu, C. Sutton, “A Survey of Machine Learning for Big Code and Naturalness,” ACM Computing Surveys, vol. 51, no. 4, pp. 1–37, 2018.

Thank You

Questions & Discussion

Project: CodeArchaeologist — AST-Guided, LLM-Powered Legacy Code Comprehension
Guide: Professor Name